Documentation Réplication MariaDB

Maître → Esclave — base glpi — Projet M2L (Maison des Ligues)



Produit par Martinière Julien

À quoi sert la réplication MariaDB ?

La réplication MariaDB permet de recopier automatiquement, en continu, les données d'un serveur de bases de données (le **maître**) vers un second serveur (l'**esclave**). Chaque modification faite sur le maître est inscrite dans un journal binaire, que l'esclave récupère puis rejoue. On obtient ainsi une copie à jour des données, qui sert à répartir la charge (les écritures sur le maître, les lectures sur l'esclave) et à disposer d'un second serveur en cas de problème sur le premier.

SOMMAIRE :

1/ Présentation et objectif.....3
Objectif.....3
Public visé.....3
2/ Architecture.....4
Lecture du schéma.....4
3/ Prérequis et environnement.....5
Tableau des machines.....5
État du maître avant le projet.....5
4/ Installation et sécurisation de l'esclave.....6
1. Vérifications préalables.....6
2. Installation de MariaDB.....6
3. Sécurisation.....6
5/ Configuration de la réplication.....7
Rôle de chaque paramètre.....7
6/ Amorçage et démarrage de la réplication.....8
1. Correction des droits côté maître.....8
2. Relever la position du journal binaire.....8
3. Export et import de la base glpi.....8
4. Démarrage de la réplication.....8
7/ Vérification et tests.....10
1. État de la réplication.....10
2. Test de robustesse après redémarrage.....10
3. Contrôle du contenu après installation de GLPI.....10
8/ Ouverture réseau et comptes applicatifs GLPI.....12
1. Ouverture de l'esclave aux connexions distantes.....12
2. Compte GLPI en lecture seule sur l'esclave.....12
3. Passage de l'esclave en lecture seule.....12
4. Compte GLPI côté maître (à prévoir).....12
Récapitulatif des comptes.....13
9/ Écarts par rapport au plan initial.....14
10/ Points de vigilance et exploitation.....15
1. Le filtre binlog-do-db.....15
2. Le routage inter-sous-réseaux.....15
3. Actions à ne jamais faire.....15
11/ Procédures utiles.....16
Vérifier l'état de la réplication.....16
Relancer la réplication après un arrêt.....16
12/ Conclusion.....17

1/ Présentation et objectif

Ce document décrit la mise en place d'une **réplication MariaDB maître → esclave** de la base `glpi`, réalisée dans le cadre du projet BTS SIO SISR sur le contexte fictif de la **M2L (Maison des Ligues)**.

Objectif

Mettre en place une architecture GLPI maître/réplica :

- ✓ les **écritures** de GLPI sont envoyées au **maître** (SGBD 1, 192.168.1.4) ;
- ✓ les **lectures** peuvent être servies par l'**esclave** (SGBD 2, 192.168.1.5), qui reçoit en continu une copie de la base `glpi`.

Public visé

Ce document s'adresse au jury du BTS SIO ainsi qu'à toute personne chargée de l'exploitation technique de la plateforme.

> **Note sur les identifiants**
>
> Les mots de passe réels sont masqués dans ce document et remplacés par `<MDP_REPLICATION>` (compte de réplication) et `<MDP_GLPI>` (compte applicatif GLPI).
>
> Il s'agit d'identifiants de **laboratoire**, utilisés uniquement dans un environnement de formation.

2/ Architecture

L'architecture repose sur trois machines. Le serveur applicatif GLPI est **volontairement placé sur un sous-réseau différent** de celui des bases de données (segmentation réseau).

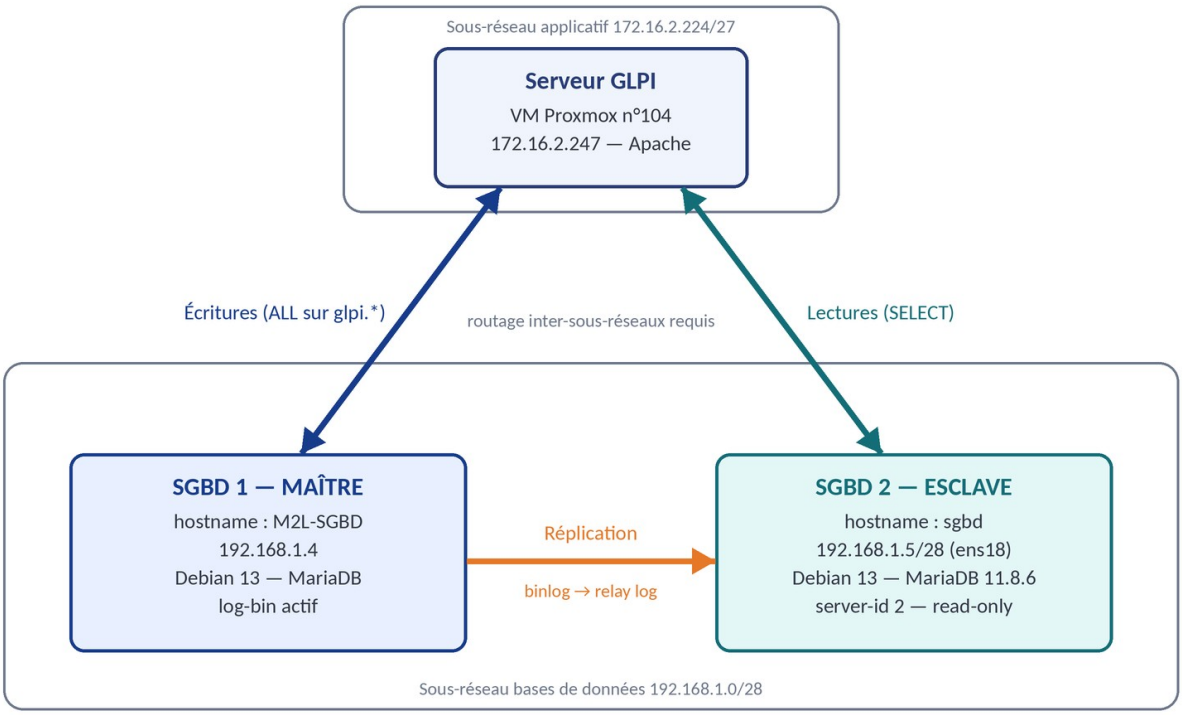


Figure 1 – Architecture GLPI maître / réplica

Lecture du schéma

- ✓ **Serveur GLPI → maître** : toutes les écritures (création de tickets, saisie du parc…).
- ✓ **Serveur GLPI → esclave** : lectures uniquement (compte en `SELECT`).
- ✓ **Maître → esclave** : réplication de la base `glpi` via le journal binaire (binlog) du maître, rejoué par l'esclave à partir de son relay log.

3/ Prérequis et environnement

Tableau des machines

Rôle	Hostname	Adresse IP	OS	MariaDB
Serveur applicatif GLPI (VM Proxmox n°104)	—	172.16.2.247/27	—	— (Apache, HTTP 200)
SGBD 1 — Maître	M2L-SGBD	192.168.1.4	Debian 13	MariaDB
SGBD 2 — Esclave	sgbd	192.168.1.5/28 (ens18)	Debian 13	11.8.6

État du maître avant le projet

Le maître était **déjà configuré en amont** pour la réplication :

- ✓ journal binaire activé (`log-bin`) ;
- ✓ `binlog-do-db = glpi` (ainsi que `zoo`) ;
- ✓ `bind-address = 0.0.0.0` (écoute sur toutes les interfaces).

Le travail décrit dans ce document porte donc principalement sur **l'esclave (192.168.1.5)**, avec quelques corrections côté maître.

4/ Installation et sécurisation de l'esclave

1. Vérifications préalables

Avant toute installation, on contrôle le nom de la machine, son adressage IP, l'accès au maître et l'accès à Internet (dépôts Debian).

julien@sgbd : vérifications préalables

```
julien@sgbd:~$ hostname
sgbd
julien@sgbd:~$ ip -brief address show ens18
ens18            UP             192.168.1.5/28
julien@sgbd:~$ ping -c 2 192.168.1.4
# → le maître répond : connectivité OK
julien@sgbd:~$ ping -c 2 deb.debian.org
# → réponse OK : accès aux dépôts Debian
```

Figure 2 – Vérification du hostname, de l'IP et de la connectivité

2. Installation de MariaDB

On installe le paquet `mariadb-server` ; la version obtenue est **MariaDB 11.8.6** et le service est actif.

julien@sgbd : installation de MariaDB

```
julien@sgbd:~$ sudo apt update
julien@sgbd:~$ sudo apt install -y mariadb-server
julien@sgbd:~$ mariadb --version
# → version installée : MariaDB 11.8.6
julien@sgbd:~$ systemctl is-active mariadb
active
```

Figure 3 – Installation du serveur MariaDB

3. Sécurisation

La sécurisation a été faite de manière **non interactive**, en exécutant directement les requêtes équivalentes à `mariadb-secure-installation` :

1. Suppression des comptes anonymes.
2. Compte `root` autorisé **uniquement en localhost**.
3. Suppression de la base de démonstration `test`.
4. Rechargement des droits avec `FLUSH PRIVILEGES`.

julien@sgbd : sécurisation (équivalent mariadb-secure-installation)

```
julien@sgbd:~$ sudo mysql
MariaDB [(none)]> DELETE FROM mysql.global_priv WHERE User='';
MariaDB [(none)]> DELETE FROM mysql.global_priv WHERE User='root'
    -> AND Host NOT IN ('localhost','127.0.0.1','::1');
MariaDB [(none)]> DROP DATABASE IF EXISTS test;
MariaDB [(none)]> DELETE FROM mysql.db WHERE Db='test' OR Db='test\_%';
MariaDB [(none)]> FLUSH PRIVILEGES;
-- root reste en authentification unix_socket : connexion via « sudo mysql »
```

Figure 4 – Sécurisation de l'instance MariaDB

L'authentification **unix_socket** de `root` a été conservée : on se connecte en administrateur avec `sudo mysql`, sans mot de passe stocké.

5/ Configuration de la réplication

Plutôt que de modifier le fichier principal, on crée un fichier dédié, lu automatiquement par MariaDB : `/etc/mysql/mariadb.conf.d/99-replication.cnf`.



Figure 5 – Contenu du fichier 99-replication.cnf

Rôle de chaque paramètre

Paramètre	Valeur	Rôle
`server-id`	2	Identifiant unique de l'esclave (doit être différent de celui du maître).
`replicate-do-db`	glpi	Seule la base glpi est répliquée ; tout le reste est ignoré.
`relay-log`	/var/lib/mysql/mysql-relay-bin	Journal relais : copie locale des événements reçus du maître.
`relay-log-index`	/var/lib/mysql/mysql-relay-bin.index	Index des fichiers du journal relais.
`log-error`	/var/lib/mysql/mysql.err	Journal d'erreurs de MariaDB.
`master-info-file`	/var/lib/mysql/mysql-master.info	Mémorise les informations de connexion au maître.
`relay-log-info-file`	/var/lib/mysql/mysql-relay-log.info	Mémorise la position de lecture dans le relay log.
`log-bin`	/var/lib/mysql/mysql-bin	Journal binaire propre à l'esclave.

Après enregistrement du fichier, on redémarre MariaDB pour prendre en compte la configuration.

6/ Amorçage et démarrage de la réplication

1. Correction des droits côté maître

Au départ, le compte `root@'192.168.1.5'` ne disposait que du privilège `REPLICATION SLAVE`. Cela suffisait pour répliquer, mais **empêchait l'export** (`mysqldump`) et la commande `SHOW MASTER STATUS`. Les droits ont donc été élargis sur le maître :

root@M2L-SGBD : correction des droits côté maître

```
root@M2L-SGBD:~$ sudo mysql
MariaDB [(none)]> GRANT ALL PRIVILEGES ON *.* TO 'root'@'192.168.1.5'
    -> WITH GRANT OPTION;
MariaDB [(none)]> FLUSH PRIVILEGES;
-- avant : seulement REPLICATION SLAVE → mysqldump et SHOW MASTER STATUS refusés
```

Figure 6 – Élargissement des droits du compte root@192.168.1.5 sur le maître

`GRANT ALL PRIVILEGES … WITH GRANT OPTION` inclut notamment le privilège `BINLOG MONITOR`, nécessaire pour lire l'état du journal binaire.

2. Relever la position du journal binaire

root@M2L-SGBD : position du journal binaire

```
MariaDB [(none)]> SHOW MASTER STATUS;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000002 |      342 | glpi,zoo     |                  |
+------------------+----------+--------------+------------------+
-- File et Position sont à reporter dans CHANGE MASTER TO
```

Figure 7 – Position du binlog relevée sur le maître

Information	Valeur relevée
MASTER_LOG_FILE	mysql-bin.000002
MASTER_LOG_POS	342

3. Export et import de la base glpi

Pour que l'esclave parte du même état que le maître, la base est exportée depuis le maître (`mysqldump --databases glpi`) puis importée sur l'esclave. À ce moment, `glpi` était encore **vide** (création de la base, 0 table).

julien@sgbd : amorçage — export depuis le maître puis import

```
julien@sgbd:~$ mysqldump -h 192.168.1.4 -u root -p --databases glpi > m2l.sql
Enter password: <MDP_REPLICATION>
julien@sgbd:~$ sudo mysql < m2l.sql
julien@sgbd:~$ sudo mysql -e "SHOW DATABASES LIKE 'glpi';"
glpi
# glpi était encore vide à ce stade (CREATE DATABASE, 0 table)
```

Figure 8 – Export de glpi depuis le maître et import sur l'esclave

4. Démarrage de la réplication

On indique à l'esclave où se trouve le maître et à partir de quelle position lire le journal, puis on démarre les threads de réplication :



Figure 9 – CHANGE MASTER TO puis START SLAVE

7/ Vérification et tests

1. État de la réplication

julien@sgbd : SHOW SLAVE STATUS (extrait)

```
MariaDB [(none)]> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Master_Host: 192.168.1.4
               Master_User: root
          Slave_IO_Running: Yes
         Slave_SQL_Running: Yes
           Replicate_Do_DB: glpi
             Last_IO_Error:
            Last_SQL_Error:
      Seconds_Behind_Master: 0
-- extrait : seuls les champs utiles au contrôle sont affichés
```

Figure 10 – Résultat de SHOW SLAVE STATUS

Champ	Valeur obtenue	Signification
Slave_IO_Running	Yes	L'esclave reçoit bien le journal du maître.
Slave_SQL_Running	Yes	L'esclave rejoue bien les requêtes reçues.
Last_IO_Error / Last_SQL_Error	(vides)	Aucune erreur de réplication.
Replicate_Do_DB	glpi	Le filtre de réplication est actif.
Seconds_Behind_Master	0	Aucun retard sur le maître.

2. Test de robustesse après redémarrage

MariaDB a été redémarré sur l'esclave : la réplication **repart seule**, avec les mêmes indicateurs au vert.

julien@sgbd : test de robustesse après redémarrage

```
julien@sgbd:~$ sudo systemctl restart mariadb
julien@sgbd:~$ sudo mysql -e "SHOW SLAVE STATUS\G" | grep -E "Running|Behind"
          Slave_IO_Running: Yes
         Slave_SQL_Running: Yes
      Seconds_Behind_Master: 0
# la réplication repart seule, sans intervention
```

Figure 11 – La réplication reprend automatiquement après redémarrage

3. Contrôle du contenu après installation de GLPI

Après l'installation applicative de GLPI (création du schéma sur le maître), la base `glpi` contient **442 tables**. Elles ont été répliquées **intégralement** vers l'esclave, et le **dernier enregistrement est identique** des deux côtés.

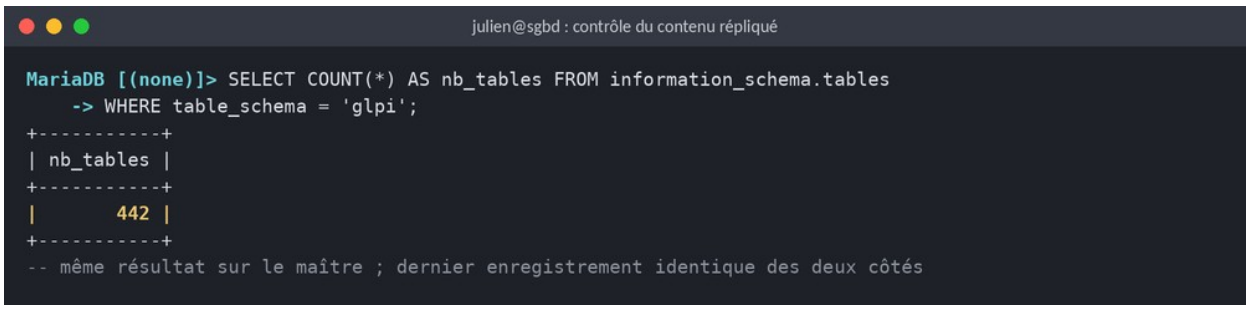


Figure 12 – Nombre de tables de la base glpi sur l'esclave

Résultat

La réplication maître → esclave de la base `glpi` est **opérationnelle**, sans erreur ni retard, et survit à un redémarrage.

8/ Ouverture réseau et comptes applicatifs GLPI

1. Ouverture de l'esclave aux connexions distantes

Par défaut, MariaDB n'écoute qu'en local. Pour que le serveur GLPI puisse lire sur l'esclave, `bind-address` passe de `127.0.0.1` à `0.0.0.0`. L'écoute sur `0.0.0.0:3306` a été confirmée.

julien@sgbd : ouverture aux connexions distantes

```
julien@sgbd:~$ sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
#   bind-address = 127.0.0.1   →   bind-address = 0.0.0.0
julien@sgbd:~$ sudo systemctl restart mariadb
julien@sgbd:~$ sudo ss -tlnp | grep 3306
LISTEN 0  80  0.0.0.0:3306  0.0.0.0:*  users:(("mariadbd",...))
```

Figure 13 – Modification de bind-address et contrôle de l'écoute sur le port 3306

2. Compte GLPI en lecture seule sur l'esclave

Le compte applicatif `'glpi'@'172.16.2.247'` ne reçoit que le droit `SELECT` sur `glpi.*` : c'est le rôle normal d'un esclave, qui ne doit servir qu'aux lectures.

julien@sgbd : compte applicatif GLPI en lecture seule

```
julien@sgbd:~$ sudo mysql
MariaDB [(none)]> CREATE USER 'glpi'@'172.16.2.247' IDENTIFIED BY '<MDP_GLPI>';
MariaDB [(none)]> GRANT SELECT ON glpi.* TO 'glpi'@'172.16.2.247';
MariaDB [(none)]> FLUSH PRIVILEGES;
MariaDB [(none)]> SHOW GRANTS FOR 'glpi'@'172.16.2.247';
GRANT SELECT ON `glpi`.* TO `glpi`@`172.16.2.247`
```

Figure 14 – Création du compte glpi en lecture seule

3. Passage de l'esclave en lecture seule

Par sécurité, l'option `read-only = 1` est ajoutée sous `[mysqld]`. Après redémarrage, `@@global.read_only = 1` est confirmé.

julien@sgbd : passage de l'esclave en lecture seule

```
# ajout sous [mysqld] dans 99-replication.cnf :
read-only = 1
julien@sgbd:~$ sudo systemctl restart mariadb
julien@sgbd:~$ sudo mysql -e "SELECT @@global.read_only;"
+--------------------+
| @@global.read_only |
+--------------------+
|                  1 |
+--------------------+
```

Figure 15 – Activation et contrôle de read_only

> **Pourquoi la réplication continue malgré read_only ?**
>
> Le thread SQL de réplication n'est pas concerné par `read_only` : il continue d'appliquer les modifications venant du maître. Seuls les **comptes applicatifs** (comme `glpi`) sont bloqués en écriture.

4. Compte GLPI côté maître (à prévoir)

Pour les écritures, GLPI doit disposer sur le maître d'un compte `'glpi'@'172.16.2.247'` avec **tous les droits** sur `glpi.*`, puisque c'est le maître qui reçoit les écritures.



Figure 16 – Commandes à prévoir sur le maître pour le compte GLPI

Récapitulatif des comptes

Serveur	Compte	Droits	État
Maître 192.168.1.4	root@'192.168.1.5'	ALL PRIVILEGES WITH GRANT OPTION	Fait
Esclave 192.168.1.5	glpi@'172.16.2.247'	SELECT sur glpi.*	Fait
Maître 192.168.1.4	glpi@'172.16.2.247'	ALL PRIVILEGES sur glpi.*	À prévoir

9/ Écarts par rapport au plan initial

Plusieurs points ont été réalisés différemment de ce qui était prévu. Ils sont listés ici de manière explicite.

Prévu	Réalisé	Explication
Hostname « M2L-SGB1 »	Hostname « sgbd »	Le nom réel de la machine est sgbd ; il n'a pas été modifié.
Configuration via SSH (compte sgbdadmin + clé SSH)	Configuration en local sur la machine	Le compte sgbdadmin et la clé SSH prévus n'existaient pas.
Réplication de glpi et zoo	Réplication de glpi uniquement	zoo exclue volontairement, à la demande (replicate-do-db = glpi). Le maître garde zoo dans binlog-do-db, sans effet : l'esclave la filtre.
Test d'écriture sur zoo.garde	Non réalisé	zoo est exclue et les tests sont interdits sur glpi. Validation faite via SHOW SLAVE STATUS et le contrôle des 442 tables.

10/ Points de vigilance et exploitation

1. Le filtre binlog-do-db

Sur le maître, avec `binlog-do-db`, une requête n'est inscrite dans le journal binaire (et donc répliquée) **que si la base courante est glpi**. Avant toute écriture manuelle à répliquer, il faut donc toujours commencer par :

root@M2L-SGBD : écriture manuelle à répliquer

```
MariaDB [(none)]> USE glpi;
-- obligatoire : avec binlog-do-db, seule une requête exécutée
-- alors que la base courante est glpi est journalisée, donc répliquée
```

Figure 17 – Sélectionner la base glpi avant une écriture manuelle

2. Le routage inter-sous-réseaux

Le serveur GLPI (172.16.2.x) et les bases (192.168.1.x) sont sur des sous-réseaux différents. Un **routage entre ces deux réseaux** est indispensable pour que GLPI puisse joindre le maître et l'esclave sur le port 3306.

3. Actions à ne jamais faire

> **Risque de casser la réplication**
>
> Ne **jamais** exécuter `RESET MASTER` sur le maître.
>
> Ne **pas** activer `read_only` sur le maître (il doit recevoir les écritures).
>
> Ne **pas** modifier `server-id`, ni `log-bin` / `binlog-do-db`.

11/ Procédures utiles

Vérifier l'état de la réplication

1. Se connecter sur l'esclave avec `sudo mysql`.
2. Lancer `SHOW SLAVE STATUS\G`.
3. Contrôler que `Slave_IO_Running` et `Slave_SQL_Running` valent **Yes**.
4. Contrôler que `Last_IO_Error` et `Last_SQL_Error` sont **vides**.
5. Contrôler `Seconds_Behind_Master` (0 = aucun retard).

Relancer la réplication après un arrêt

1. Vérifier que le service MariaDB est actif (`systemctl status mariadb`).
2. Se connecter avec `sudo mysql`.
3. Exécuter `STOP SLAVE;` puis `START SLAVE;`.
4. Revérifier l'état avec `SHOW SLAVE STATUS\G`.

julien@sgbd : relancer la réplication après un arrêt

```
julien@sgbd:~$ sudo systemctl status mariadb --no-pager
julien@sgbd:~$ sudo mysql
MariaDB [(none)]> STOP SLAVE;
MariaDB [(none)]> START SLAVE;
MariaDB [(none)]> SHOW SLAVE STATUS\G
-- contrôler : Slave_IO_Running = Yes, Slave_SQL_Running = Yes,
--             Last_IO_Error / Last_SQL_Error vides
```

Figure 18 – Relance de la réplication sur l'esclave

12/ Conclusion

La réplication MariaDB maître → esclave de la base `glpi` est en place et fonctionnelle : les deux threads de réplication sont actifs, sans erreur ni retard, et la réplication redémarre seule après un redémarrage du service. Les 442 tables créées par l'installation de GLPI ont été recopiées intégralement sur l'esclave.

L'esclave est ouvert au réseau, protégé en lecture seule et dispose d'un compte applicatif limité à `SELECT`, ce qui correspond à son rôle de réplica de lecture. Il reste à créer sur le maître le compte GLPI dédié aux écritures et à s'assurer du routage entre le sous-réseau applicatif et celui des bases.

Les écarts par rapport au plan initial (nom de machine, configuration locale, exclusion de la base zoo, mode de validation) ont été documentés et n'affectent pas le fonctionnement de la réplication.